

Giải trò chơi đoán số bằng máy tính

Mo Tao
Khoa Máy tính, Đại học Thanh Hoa

Tháng 1 năm 2011

Nguồn gốc: Giải trò chơi đoán số bằng máy tính

Others / Bulls and Cows computer solving - Mo Tao

Abstract

Bài viết khảo sát nhiều chiến lược để máy tính giải trò chơi đoán số **Bulls and Cows**. Trọng tâm là cách giảm số lần đoán trung bình, tăng tốc các thuật toán heuristic, và hiện thực những thuật toán đó bằng C++. Bản dịch giữ cấu trúc bài gốc: luật chuẩn và biến thể, phương pháp lọc tập ứng viên, các chiến lược đơn giản, các hàm đánh giá heuristic, tối ưu bằng lớp tương đương, bảng kết quả thực nghiệm, cùng mô tả thư viện kiểm thử và chương trình chính.

Từ khóa. Phương pháp lọc; hàm đánh giá; tối ưu bằng lớp tương đương; Bulls and Cows; Mastermind.

Contents

1	Mở đầu	1
1.1	Luật chuẩn	1
1.2	Các biến thể	2
2	Chiến lược giải với luật chuẩn	2
2.1	Tiêu chuẩn thống nhất	2
2.2	Phương pháp lọc	3
2.3	Các chiến lược đơn giản	4
2.4	Chiến lược heuristic	5
2.5	Thiết kế hàm đánh giá	5
2.5.1	Kích thước tập ứng viên tệ nhất	5
2.5.2	Kích thước tập ứng viên trung bình	6
2.5.3	Số bước kỳ vọng	7
2.5.4	Số loại phản hồi	7
2.6	Tối ưu thuật toán	8
2.7	Mô tả mã nguồn và chương trình	9
2.7.1	Thư viện kiểm thử	9
2.7.2	Chương trình chính	10

3 Những thử nghiệm khác	11
3.1 Chiến lược tối ưu hóa	11
3.2 Trò chơi có nhiều vị trí hơn	11
4 Kết luận	12
4.1 Tổng kết	12
4.2 Hướng nghiên cứu tiếp theo	12
5 Tài liệu tham khảo	12
A Phụ lục của bài gốc	13

1 Mở đầu

Trò chơi đoán số, còn gọi là **Bulls and Cows**, là một trò chơi trí tuệ nhỏ được cho là phổ biến ở Anh từ khoảng giữa thế kỷ 20. Trò chơi thường do hai người chơi, cũng có thể là một người chơi với máy tính, và có thể chơi trên giấy hoặc trực tuyến. Luật chơi đơn giản, nhưng lại kiểm tra tính chặt chẽ và sự kiên nhẫn của người chơi.

1.1 Luật chuẩn

Thông thường một người ra số và một người đoán. Người ra số nghĩ trước một số gồm bốn chữ số không lặp lại, trong đó chữ số đầu tiên được phép là 0, và không cho người đoán biết. Mỗi lần người đoán đưa ra một số, người ra số trả lời theo dạng $xAyB$: x là số chữ số đúng và đúng vị trí, còn y là số chữ số đúng nhưng sai vị trí.

Ví dụ, nếu đáp án là 5234 và người đoán đưa ra 5346, phản hồi là 1A2B. Chữ số 5 đúng vị trí nên tính 1A; hai chữ số 3 và 4 có trong đáp án nhưng sai vị trí nên tính 2B. Người đoán tiếp tục dựa vào các phản hồi đó cho đến khi đoán trúng, tức nhận 4A0B.

Trò chơi thường có giới hạn số lần đoán. Theo tính toán bằng máy tính, nếu dùng chiến lược chặt chẽ thì với luật chuẩn mọi đáp án đều có thể đoán ra trong nhiều nhất 7 lần. Cần chú ý rằng một số nơi định nghĩa giới hạn là “sau bao nhiêu lần đoán thì đã có thể xác định chắc chắn đáp án”. Theo cách đó, đôi khi người chơi vẫn cần thêm một lần đoán nữa để thật sự nhận phản hồi 4A0B.

1.2 Các biến thể

Luật chuẩn dùng 10 chữ số 0 đến 9 và 4 vị trí. Có thể thay đổi số chữ số hoặc số vị trí để tạo biến thể, chẳng hạn dùng 9 chữ số cho trò chơi 4 vị trí.

Một biến thể khác cho phép lặp chữ số; trò chơi này thường được gọi là **Mastermind**. Ngoài luật chuẩn, khi có chữ số lặp, mỗi lần xuất hiện chỉ được ghép một lần và lấy kết quả tốt nhất. Ví dụ đáp án là 5543, đoán 5255. Không thể ghép chữ số 5 đầu tiên của lượt đoán với chữ số 5 thứ hai của đáp án nếu điều đó làm kết quả kém tối ưu. Kết quả đúng là 1A1B: chữ số 5 đầu đúng vị trí, và một trong hai chữ số 5 còn lại của lượt đoán ghép với chữ số 5 thứ hai của đáp án. Nếu đoán 5267 thì chỉ có 1A0B, vì chữ số 5 đầu đã được tính đúng vị trí và không thể dùng lại để ghép sai vị trí.

Mastermind chuẩn có 4 vị trí, mỗi vị trí lấy một trong 6 giá trị, khi chơi thường viết bằng A đến F. Các biến thể khác gồm Royale Mastermind với 5 vị trí và mỗi vị trí lấy một trong 5 giá trị, Mastermind

Challenge trong đó hai bên thay phiên ra đề cho nhau, và Word Mastermind trong đó 0 đến 25 được xem là A đến Z, còn chuỗi cần đoán phải là một từ.

Một biến thể khó hơn là **Static Game**: các lượt đoán không được phụ thuộc vào phản hồi. Người chơi đưa ra một lần một tập các lượt đoán; với mọi đáp án, các phản hồi tương ứng phải đủ để xác định đáp án. Với Mastermind chuẩn, Greenwell đã đưa ra một nghiệm chỉ gồm 6 lượt đoán:

(1, 2, 2, 1), (2, 3, 5, 4), (3, 3, 1, 1),
(4, 5, 2, 4), (5, 6, 5, 6), (6, 6, 4, 3).

Về mặt lý thuyết 5 lượt đoán là đủ, nhưng bài gốc ghi nhận rằng khi đó chưa tìm được nghiệm như vậy.

2 Chiến lược giải với luật chuẩn

Trò chơi đoán số là một bài toán tương tác thông tin cổ điển: người chơi cần dùng càng ít câu hỏi càng tốt để thu thập thông tin từ người ra số, rồi suy ra đáp án. Điểm phức tạp là các câu hỏi được đặt tuần tự; người chơi có thể dựa vào phản hồi trước đó để điều chỉnh chiến lược.

Phần này trước hết thống nhất tiêu chuẩn đánh giá, sau đó trình bày phương pháp lọc, là thuật toán phổ biến nhất cho trò chơi này. Các chiến lược đơn giản và heuristic sẽ được so sánh về cơ sở lý thuyết, hiện thực, kết quả chạy và độ phức tạp. Cuối cùng là một kỹ thuật tối ưu quan trọng cho thuật toán heuristic và phần mô tả mã nguồn.

2.1 Tiêu chuẩn thống nhất

Để so sánh các chiến lược, bài gốc dùng các quy ước sau:

1. Chơi theo luật chuẩn: 4 vị trí, mỗi vị trí lấy một chữ số trong 0 đến 9, và các chữ số khác nhau.
2. Trò chơi kết thúc khi một lượt đoán nhận 4A0B; lượt đoán này cũng được tính vào tổng số lần đoán.
3. Người ra số công bằng: không rò rỉ thông tin, không trả lời sai, và không đổi đáp án giữa chừng để gây khó cho người đoán.
4. Mọi ván phải kết thúc trong nhiều nhất 7 lần đoán; dưới ràng buộc đó, giảm số lần đoán trung bình. Kết quả được thống kê bằng cách thử toàn bộ 5040 đáp án.
5. Tối ưu thời gian chạy; mỗi ván nên chạy không quá 1 giây.

Với quy ước thứ hai, một chiến lược có thể tình cờ đoán trúng khi chưa chắc chắn, hoặc ngược lại đã xác định được đáp án nhưng vẫn phải dùng thêm một lượt đoán để nhận 4A0B. Về kỳ vọng, chi tiết này không làm thay đổi bản chất, và nó phù hợp hơn với trực giác rằng trò chơi kết thúc khi “đoán trúng”, không chỉ khi “biết đáp án”.

Quy ước thứ ba loại bỏ người ra số đối kháng. Nếu người ra số được phép chọn đáp án thích nghi nhưng vẫn giữ cho phản hồi nhất quán, họ có thể làm chiến lược của người đoán tệ đi. Ví dụ, khi người đoán đã xác định ba vị trí đầu là 012 và định lần lượt thử 0123, 0124, ..., 0129, trung bình chỉ cần $7/2 = 3.5$ lượt; nhưng một người ra số đối kháng có thể chọn 0129, buộc người đoán dùng 7 lượt. Vì vậy bài gốc giả định đáp án được cố định ngay từ đầu, hoặc được quản lý bởi một chương trình thư viện.

Quy ước thứ tư phản ánh hai mục tiêu thường gặp: giảm số lần đoán tệ nhất và giảm số lần đoán trung bình. Trong một số biến thể, hai mục tiêu không đồng thời đạt được. Chẳng hạn với Mastermind, chiến lược có trung bình tốt nhất cần khoảng 4.340 lượt nhưng trường hợp xấu nhất cần 6 lượt; nếu buộc nhiều nhất 5 lượt, trung bình tốt nhất tăng lên khoảng 4.341. Với luật chuẩn Bulls and Cows, hầu hết chiến lược trung bình tốt cũng chỉ cần 7 lượt trong trường hợp xấu nhất, nên ràng buộc này là hợp lý.

Với quy ước thứ năm, nếu bỏ giới hạn thời gian thì máy tính có thể duyệt toàn bộ cây trò chơi hữu hạn để tìm chiến lược tối ưu. Tuy nhiên, ngay cả khi mỗi ván chỉ mất 1 giây, thử hết 5040 đáp án cũng mất 5040 giây, tức hơn 1 giờ 24 phút. Do đó tối ưu thời gian chạy cũng trực tiếp giúp thử nghiệm và điều chỉnh chiến lược tốt hơn. Bài gốc đưa ra một tối ưu giúp giảm thời gian chạy từ hơn 3000 giây xuống dưới 100 giây.

2.2 Phương pháp lọc

Với luật chuẩn, số đáp án chỉ là

$$10 \cdot 9 \cdot 8 \cdot 7 = 5040.$$

Với Mastermind chuẩn là $6^4 = 1296$, với Royale Mastermind là $5^5 = 3125$. Số khả năng nhỏ là một đặc điểm tự nhiên của trò chơi: nếu không, người chơi người thật khó có thể suy luận thủ công.

Một đặc điểm khác là miền giá trị ở mỗi vị trí không quá lớn. Nếu mỗi vị trí có thể là 0 đến 99, chỉ 2 vị trí đã có $100^2 = 10000$ khả năng. Trực giác khi con người giải trò chơi này thường là trước hết xác định các chữ số xuất hiện trong đáp án, cố gắng nhận phản hồi kiểu 1A3B hoặc 2A2B, rồi hoán vị chúng để tìm đúng vị trí. Miền chữ số quá rộng làm cách suy luận này kém hiệu quả.

Có hai hướng thiết kế thuật toán. Hướng thứ nhất mô phỏng kỹ thuật của con người bằng một quy trình đoán và suy luận logic. Cách này khó lập trình và kết quả không ổn định, vì nó chưa khai thác trực tiếp đặc điểm bản chất: tập kết quả có thể là hữu hạn và nhỏ. Hướng thứ hai là **phương pháp lọc**, được dùng xuyên suốt bài này.

Thuật toán lọc:

1. Khởi tạo tập ứng viên $\mathcal{S}$ gồm toàn bộ 5040 đáp án có thể.
2. Dùng một chiến lược nào đó để chọn lượt đoán X , nhận phản hồi $R = xAyB$.
3. Xóa khỏi $\mathcal{S}$ mọi đáp án Y mà nếu đáp án là Y thì đoán X không cho phản hồi R .
4. Nếu $\mathcal{S}$ còn hơn một đáp án, quay lại bước 2.
5. Ứng viên duy nhất còn lại là đáp án.

Mã giả:

```
Solve()
  S <- all possible answers
  while |S| > 1 do
    X <- find(S)
    R <- feedback for guess X
    for every Y in S do
      if feedback(answer = Y, guess = X) != R then
        remove Y from S
      end if
    end for
  end while
  return the only element of S
end Solve
```

Tên “lọc” đến từ việc mỗi phản hồi loại bỏ các đáp án không còn nhất quán. Bước then chốt là hàm `find`: từ tập ứng viên hiện tại, chọn lượt đoán sao cho trung bình cần ít lượt đoán nhất.

2.3 Các chiến lược đơn giản

Một ý tưởng trực quan là chỉ đoán các phần tử trong tập ứng viên. Như vậy bất kể phản hồi ra sao, lượt đoán vẫn có xác suất trúng đáp án. Câu hỏi còn lại là chọn phần tử nào trong $\mathcal{S}$.

Chiến lược thứ nhất sắp $\mathcal{S}$ theo thứ tự từ điển và chọn phần tử nhỏ nhất:

```
find(S)
  sort S in lexicographic order
  return first element of S
end find
```

Với hai chuỗi số khác nhau A, B , tìm vị trí nhỏ nhất p sao cho $A_p \neq B_p$; nếu $A_p < B_p$ thì A nhỏ hơn B theo thứ tự từ điển.

Kết quả:

Số lần đoán trung bình	5.560
Số lần đoán nhiều nhất	9
Tổng thời gian	10.367 giây

Số lần	1	2	3	4	5	6	7	8	9
Số ván	1	13	108	596	1668	1768	752	129	5

Chiến lược thứ hai chọn ngẫu nhiên đều một phần tử của $\mathcal{S}$:

```
find(S)
  return a uniformly random element of S
end find
```

Kết quả:

Số lần đoán trung bình	5.465
Số lần đoán nhiều nhất	9
Tổng thời gian	10.508 giây

Số lần	1	2	3	4	5	6	7	8	9
Số ván	1	10	114	593	1844	1804	626	47	1

Chiến lược ngẫu nhiên tốt hơn rõ rệt: số ván cần hơn 7 lượt giảm mạnh. Dù vậy, cả hai đều không thỏa quy ước “nhiều nhất 7 lượt”. Ưu điểm của chúng là thời gian chạy chỉ khoảng 10 giây cho toàn bộ thử nghiệm.

2.4 Chiến lược heuristic

Chiến lược đơn giản kém vì lượt đoán còn quá tùy ý. Ta có thể duyệt mọi chuỗi số hợp lệ làm lượt đoán, rồi chọn chuỗi “tốt nhất”.

Vấn đề thứ nhất là định nghĩa “tốt nhất”. Duyệt toàn bộ phần còn lại của cây trò chơi để chọn lượt có số lần đoán nhỏ nhất là lý tưởng, nhưng không khả thi về thời gian. Vì vậy ta dùng **hàm đánh giá**: một hàm dùng thông tin hữu hạn hiện có để ước lượng chất lượng của một hành động và biểu diễn chất lượng đó bằng một con số. Trong trò chơi đoán số, một đánh giá tự nhiên là kích thước tập ứng viên còn lại sau lượt đoán: tập càng nhỏ thì thường cần ít lượt đoán hơn.

Vấn đề thứ hai là có nên chỉ duyệt các phần tử của $\mathcal{S}$ hay không. Với mục tiêu thu thập thông tin, một chuỗi không thể là đáp án vẫn có thể là lượt đoán tốt hơn. Nếu hàm đánh giá đủ hợp lý, miễn duyệt

rộng hơn thường cho kết quả tốt hơn. Bài gốc dùng cách dung hòa: duyệt mọi đáp án hợp lệ trong $\mathcal{A}$, và khi giá trị đánh giá bằng nhau thì ưu tiên phần tử thuộc $\mathcal{S}$.

```
find(S)
  ret <- 0123
  for every valid answer X in A do
    if value(X) < value(ret) then ret <- X
    if value(X) = value(ret) and X is in S then ret <- X
  end for
  return ret
end find
```

Quy ước là $\text{value}(X)$ càng nhỏ thì lượt đoán X càng tốt.

2.5 Thiết kế hàm đánh giá

2.5.1 Kích thước tập ứng viên tề nhất

Hàm đánh giá chỉ dựa trên tập ứng viên hiện tại $\mathcal{S}$ và lượt đoán đang xét X . Với một lượt đoán X , phản hồi có 15 loại:

0A0B, 0A1B, 0A2B, 0A3B, 0A4B,
1A0B, 1A1B, 1A2B, 1A3B,
2A0B, 2A1B, 2A2B,
3A0B, 3A1B, 4A0B.

Gọi $\mathcal{S}[a][b]$ là tập ứng viên còn lại nếu đoán X và nhận phản hồi $aAbB$. Hàm đánh giá theo kích thước tề nhất là:

```
value(X)
  compute S[a][b]
  ret <- 0
  for a = 0..4 do
    for b = 0..4-a do
      ret <- max(ret, |S[a][b]|)
    end for
  end for
  return ret
end value
```

Kết quả:

Số lần đoán trung bình	5.389
Số lần đoán nhiều nhất	7
Tổng thời gian	79.620 giây

Số lần	1	2	3	4	5	6	7	8	9
Số ván	1	3	44	519	2106	2152	215	0	0

Chiến lược này thỏa ràng buộc 7 lượt. Trung bình giảm từ 5.465 xuống 5.389, tức trong 5040 ván, xấp xỉ cứ 10 ván thì tiết kiệm được một lượt so với chiến lược đơn giản. Đây là cải thiện đáng kể, đồng thời cho thấy trò chơi khá khó: sau khi đã có thuật toán hợp lý, cải thiện nhỏ cũng không dễ.

Nếu hiện thực ngây thơ, với mỗi X ta quét mọi phần tử của $\mathcal{S}$, tính phản hồi và phân loại vào $\mathcal{S}[a][b]$. Một quyết định có độ phức tạp $O(5040 \cdot |\mathcal{S}| \cdot |R|)$. Thử mọi ván có độ phức tạp xấp xỉ $O(|\mathcal{S}|^3)$ với hằng số lớn. Chạy trực tiếp mất hơn một giờ; các thời gian trong bài đã dùng tối ưu ở mục sau.

2.5.2 Kích thước tập ứng viên trung bình

Thay vì tối thiểu hóa trường hợp xấu nhất, ta có thể tối thiểu hóa kích thước kỳ vọng của tập còn lại. Xét ví dụ chỉ có hai vị trí, tại một thời điểm $\mathcal{S} = \{12, 21, 34\}$. Nếu đoán 34, đáp án 34 cho phản hồi 2A0B, còn 12 và 21 cho 0A0B. Kích thước còn lại kỳ vọng là

$$1 \cdot \frac{1}{3} + 2 \cdot \frac{2}{3} = \frac{5}{3}.$$

Nếu đoán 12, ba đáp án lần lượt cho ba phản hồi khác nhau 2A0B, 0A2B, 0A0B, nên kích thước còn lại kỳ vọng là 1. Vì vậy đoán 12 tốt hơn.

Hàm đánh giá:

```
value(X)
  compute S[a][b]
  ret <- 0
  for a = 0..4 do
    for b = 0..4-a do
      ret <- ret + |S[a][b]|^2
    end for
  end for
  return ret
end value
```

Giá trị kỳ vọng thật sự còn chia cho $|\mathcal{S}|$, nhưng trong cùng một quyết định mẫu số không đổi nên có thể bỏ qua.

Kết quả:

Số lần đoán trung bình	5.268
Số lần đoán nhiều nhất	7
Tổng thời gian	73.762 giây

Số lần	1	2	3	4	5	6	7	8	9
Số ván	1	4	59	573	2430	1886	87	0	0

Trung bình tiếp tục cải thiện rõ rệt, trong khi các số liệu khác gần như không đổi. Hàm đánh giá này khớp tốt hơn với chất lượng thực tế của lượt đoán.

2.5.3 Số bước kỳ vọng

Một hướng tự nhiên hơn là dùng kỳ vọng số lượt đoán còn lại:

```
value(X)
  compute S[a][b]
  ret <- 0
  for a = 0..4 do
    for b = 0..4-a do
```

```

        ret <- ret + |S[a][b]| * Step(S[a][b])
    end for
end for
return ret
end value

```

Ở đây $\text{Step}(\mathcal{S}[a][b])$ là số bước kỳ vọng để giải tiếp từ tập đó. Vì không thể tính chính xác trong giới hạn thời gian, bài gốc ước lượng rằng mỗi lượt làm tập ứng viên thu nhỏ theo một tỷ lệ trung bình k , tức

$$\text{Step}(\mathcal{S}[a][b]) = \log_k |\mathcal{S}[a][b]|.$$

Giá trị cụ thể của k không ảnh hưởng đến thứ tự so sánh, nên có thể dùng

$$\text{Step}(\mathcal{S}[a][b]) = \ln |\mathcal{S}[a][b]|.$$

Kết quả:

Số lần đoán trung bình	5.265
Số lần đoán nhiều nhất	7
Tổng thời gian	95.682 giây

Số lần	1	2	3	4	5	6	7	8	9
Số ván	1	4	53	560	2515	1796	111	0	0

So với hàm kích thước trung bình, kết quả không khác nhiều. Tuy nhiên cách nhìn qua hàm Step có khả năng mở rộng tốt hơn và sẽ xuất hiện lại trong phần chiến lược tối ưu hóa.

2.5.4 Số loại phản hồi

Kooi đề xuất năm 2005 một chiến lược dựa trên ý tưởng sau: tập còn lại nhỏ hơn thì thuận lợi hơn cho những lượt sau, nhưng xác suất nó xảy ra cũng nhỏ hơn; vì vậy có thể xem đóng góp của mỗi nhánh phản hồi là như nhau. Do đó với một lượt đoán X , càng có nhiều tập còn lại khác rỗng, tức càng có nhiều loại phản hồi có thể xuất hiện, thì X càng tốt.

```

value(X)
  compute S[a][b]
  ret <- 0
  for a = 0..4 do
    for b = 0..4-a do
      if S[a][b] is not empty then ret <- ret + 1
    end for
  end for
  return -ret
end value

```

Vì value được tối thiểu hóa còn số loại phản hồi cần tối đa hóa, ta trả về giá trị đối.

Kết quả:

Số lần đoán trung bình	5.308
Số lần đoán nhiều nhất	8
Tổng thời gian	78.632 giây

Số lần	1	2	3	4	5	6	7	8	9
Số ván	1	11	80	556	2277	1929	183	3	0

Với luật chuẩn, hàm này kém hơn: có 3 ván cần 8 lượt và trung bình cao hơn. Nhưng với Mastermind, đây là một trong các chiến lược tốt đã biết. Điều này cho thấy cùng một thuật toán có thể khác biệt lớn khi đổi luật chơi.

2.6 Tối ưu thuật toán

Nút thắt là tính các tập $\mathcal{S}[a][b]$. Thuật toán ngây thơ duyệt từng lượt đoán X , quét $\mathcal{S}$, tính phản hồi và phân loại. Độ phức tạp khó giảm về bậc, nhưng có thể giảm rất mạnh hằng số.

Trong một ván, thời gian chủ yếu nằm ở vài lượt đầu, khi $\mathcal{S}$ còn lớn. Về cuối ván, $|\mathcal{S}|$ gần như là hằng số nhỏ. Ví dụ với hàm số bước kỳ vọng, khi đáp án là 9876, một quá trình có thể là:

Lượt	Đoán và phản hồi	$ \mathcal{S} $	Số X khác bản chất
1	0123, 0A0B	5040	1
2	4567, 0A2B	360	209
3	8975, 1A2B	84	3360
4	7948, 0A3B	21	5040
5	9876, 4A0B	5	5040

Ở đầu ván, nhiều lượt đoán là tương đương về bản chất. Lượt đầu, mọi X hợp lệ đều không khác nhau; có thể luôn đoán 0123 mà không ảnh hưởng đến số lượt trung bình. Lượt thứ hai, do các chữ số 4, 5, 6, 7, 8, 9 đều chưa từng xuất hiện, các lượt như 0142 và 0172 cũng tương đương.

Bài gốc dùng quy tắc sinh đại diện lớp tương đương như sau. Khi duyệt X , các chữ số chưa từng dùng phải xuất hiện theo thứ tự tăng dần. Cụ thể, nếu chữ số P chưa xuất hiện trong các lượt đoán trước, thì X chỉ được chứa P khi $P - 1$ đã xuất hiện trong lượt đoán trước đó hoặc đã xuất hiện ở phần đầu của chính X . Nhờ vậy mỗi lớp tương đương chỉ được duyệt một lần.

```

Dfs(i)
  if i = 4 then
    output one essentially different X
  else
    for P = 0..9 do
      if P satisfies the representative rule then
        Dfs(i + 1)
      end if
    end for
  end if
end Dfs

```

Hiệu quả rất lớn. Trong ví dụ trên, lượt tính nặng nhất là lượt thứ ba:

$$|\mathcal{S}| \cdot \#X = 84 \cdot 3360 = 282240,$$

nhỏ hơn rất nhiều so với ước lượng ngây thơ $5040 \cdot 5040 = 25401600$.

Một tối ưu hằng số khác nằm ở cách tính phản hồi. Nếu duyệt X ở vòng ngoài, mỗi phần tử của $\mathcal{S}$ lại phải dựng lại mảng đánh dấu chữ số để đếm B . Nếu lưu trước các X đại diện, rồi đặt vòng lặp trên $\mathcal{S}$ ở ngoài và quét các X ở trong, có thể tái sử dụng thông tin đánh dấu và giảm đáng kể chi phí. Các chi tiết hiện thực khác cũng ảnh hưởng nhiều, nhưng bài gốc không triển khai hết vì quá vụn. Sau các tối ưu này, thời gian chạy giảm xuống khoảng 100 giây.

Về bản chất, phương pháp lọc heuristic là một tìm kiếm độ sâu 1. Các kỹ thuật cắt tỉa thường dùng trong tìm kiếm đều có thể áp dụng để tối ưu nó.

2.7 Mô tả mã nguồn và chương trình

Các chương trình trong bài gốc được viết bằng C++ và chạy trên Windows 7 với Dev-C++ 4.9.9.2. Thời gian thực nghiệm được đo trên máy cá nhân có RAM 2.00 GB, CPU Intel Core i3 2.53 GHz.

2.7.1 Thư viện kiểm thử

Bài gốc viết một lớp game để quản lý toàn bộ 5040 ván và chặn truy cập trực tiếp vào đáp án, nhằm giữ công bằng. Cách dùng:

1. Đặt mã vào game.h và thêm `#include "game.h"` trong chương trình chính.
2. Đổi biến `AIname` để đổi tên các tệp xuất log.
3. Gọi `guess(a)` để đoán, trong đó `a` là mảng hoặc vector gồm 4 chữ số. Hàm trả về `pair<int, int>` chứa phản hồi (A, B) . Khi đoán trúng $4A0B$, thư viện tự chuyển sang ván tiếp theo. Nếu quá giới hạn mà chưa đoán được, ván được tính thất bại. Sau khi chạy hết 5040 ván, thư viện xuất log chi tiết và thống kê tổng.

Lỗi thư viện có các hằng số và giao diện sau:

```
const int L = 4, S = 10, maxt = 9, Size = 5040;
const pair<int, int> error = make_pair(-1, -1);
const pair<int, int> over = make_pair(-2, -2);

string AIname = "Noname";

class game {
    int P, Ps, T[maxt + 1];
    int a[L], t;
public:
    game();
    pair<int, int> guess(int* b);
};

pair<int, int> guess(int* b);
pair<int, int> guess(vector<int> b);
```

Trong `guess`, thư viện kiểm tra chữ số có nằm trong miền hợp lệ và có bị lặp hay không, tính số A bằng so sánh cùng vị trí, tính số chữ số chung rồi trừ A để nhận B . Khi một ván kết thúc, thư viện cập nhật bảng `T[i]`: số ván đoán đúng sau i lượt. Khi hết mọi đáp án, thư viện in số lần đoán trung bình, số lần nhiều nhất, tổng thời gian, và phân bố số ván theo số lượt đoán.

2.7.2 Chương trình chính

Chương trình chính trong bài gốc dùng hàm đánh giá

$$\sum_{a,b} |\mathcal{S}[a][b]| \ln(|\mathcal{S}[a][b]| + 1),$$

tức biến thể tối ưu của hàm số bước kỳ vọng. Mã dùng vector để lưu các chuỗi số. Những phần chính là:

- dfs: sinh toàn bộ 5040 đáp án hợp lệ.

- dfs2: sinh các lượt đoán khác bản chất theo quy tắc lớp tương đương.
- calc: tính phản hồi nếu một chuỗi là đáp án và một chuỗi là lượt đoán.
- find: chọn lượt đoán tốt nhất bằng cách tính bảng $ct[i][j][k]$, trong đó $ct[i][j][k]$ là kích thước tập ứng viên khi lượt đoán $h[i]$ trả về $jAkB$.
- main: lặp qua toàn bộ 5040 đáp án, duy trì tập ứng viên hiện tại, gọi guess, rồi lọc lại tập ứng viên bằng calc.

Đoạn lựa chọn hàm đánh giá có dạng:

```
double best = 1e100;
vector<int> ret;

for each candidate guess h[i] do
    double v = 0;
    for j = 0..L do
        for k = 0..L-j do
            v += ct[i][j][k] * log(ct[i][j][k] + 1);

            // Other evaluators used in the paper:
            // v += ct[i][j][k] * ct[i][j][k];
            // if (ct[i][j][k]) v += ct[i][j][k] * log(ct[i][j][k]);
            // v = max(v, double(ct[i][j][k]));
            // if (ct[i][j][k]) --v;
        end for
    end for

    if v < best, or v == best and h[i] is still possible, choose h[i]
end for
```

Trong vòng chơi chính:

```
for every real answer do
    g <- all possible answers
    t <- 0
    p <- L - 1
    repeat
        t <- t + 1
        a <- find()
        ret <- guess(a)
        if ret.first == 4 then break

        update p by the largest digit used in a
        h <- empty
        for every y in g do
            if calc(y, a) == ret then append y to h
        end for
        g <- h
    forever
end for
```

Các hàm đánh giá khác có thể kiểm thử bằng cách thay phần tính v ; chiến lược đơn giản chỉ cần thay hàm `find`.

3 Những thử nghiệm khác

Trong quá trình viết bài, khi hiệu quả trung bình của heuristic gần chạm nút thắt, tác giả tiếp tục suy nghĩ về cách cải thiện bản chất chiến lược và có một số kết quả sơ bộ. Ngoài ra, dựa trên bài guess của Winter Camp 2006, tác giả cũng thử một số luật chơi không chuẩn.

3.1 Chiến lược tối ưu hóa

Chiến lược tối ưu đầy đủ khó tính trong thực tế, nhưng tối ưu hàm đánh giá “số bước kỳ vọng” có thể đưa ta đến khá gần. Trong mục trước, $\text{Step}(\mathcal{S}[a][b]) = \ln|\mathcal{S}[a][b]|$ khá hợp lý khi tập còn lại lớn. Nhưng khi tập nhỏ, phân bố cụ thể của các phần tử trong tập ảnh hưởng rất mạnh đến số lượt cần thiết, nên sai số của xấp xỉ này có thể làm chiến lược kém đi.

Do $\mathcal{S}[a][b]$ nhỏ, về nguyên tắc có thể duyệt toàn bộ tương lai từ tập đó và dùng cắt tỉa tốt để tính chính xác $\text{Step}(\mathcal{S}[a][b])$. Bài gốc không hiện thực hướng này vì độ khó lập trình cao.

Một hướng khác là học một hàm Step chính xác hơn theo $|\mathcal{S}[a][b]|$, chẳng hạn kết hợp các hàm cơ sở như $\ln|\mathcal{S}[a][b]|$ và $|\mathcal{S}[a][b]|^k$. Ta chạy thử, thống kê với mỗi kích thước tập ứng viên cần bao nhiêu lượt đoán, dùng thống kê đó xây dựng hàm Step mới, rồi lặp lại. Đây là tinh thần của thuật toán di truyền.

Tác giả hiện thực một phiên bản đơn giản của quá trình này: mỗi vòng lặp xây dựng hàm Step kế tiếp từ hàm trước. Kết quả thực tế không quá tốt: sau khoảng 3 giờ và hơn 100 vòng lặp, chỉ thu được hàm có số lượt trung bình 5.250.

Trong lúc điều chỉnh, tác giả tìm được hàm

$$\text{Step}(\mathcal{S}[a][b]) = \ln(|\mathcal{S}[a][b]| + 1),$$

cho số lượt trung bình 5.243, tốt hơn mọi chiến lược trước đó. Một lý do có thể là khi tập còn lại nhỏ, hàm này khớp thực tế hơn.

3.2 Trò chơi có nhiều vị trí hơn

Khi số vị trí tăng, số đáp án có thể tăng rất nhanh. Với luật không lặp và 10 vị trí dùng chữ số 0 đến 9, có $10! = 3628800$ đáp án. Khi đó việc tính $\mathcal{S}[a][b]$ trở nên không khả thi, nên các heuristic ở trên không dùng trực tiếp được.

Một cách xử lý tốt là phân trường hợp. Khi tập ứng viên còn lớn, dùng chiến lược đơn giản; khi tập đã giảm đủ nhỏ, chuyển sang heuristic hoặc thậm chí chiến lược tối ưu. Lý do là ở giai đoạn đầu, số lượt đoán khác bản chất khá ít, lựa chọn khả thi không nhiều, và chất lượng các lượt đoán cũng không khác nhau quá lớn. Như đã thấy trong phần tối ưu, cách phân đoạn như vậy có thể tiết kiệm rất nhiều thời gian chạy.

4 Kết luận

4.1 Tổng kết

Bài viết trình bày phương pháp lọc, thuật toán phổ biến nhất để máy tính giải trò chơi đoán số, cùng nhiều chiến lược trên nền thuật toán đó: chiến lược đơn giản, chiến lược heuristic và các thử nghiệm tối ưu hóa. Bảng sau gom các số liệu chính.

Nhóm	Chiến lược	Nhiều nhất	Trung bình	Thời gian
Đơn giản	Nhỏ nhất theo từ điển	9	5.560	10.367 giây
Đơn giản	Ngẫu nhiên	9	5.465	10.508 giây
Heuristic	Tập tẻ nhất	7	5.389	79.620 giây
Heuristic	Tập trung bình	7	5.268	73.762 giây
Heuristic	Số bước kỳ vọng	7	5.265	95.682 giây
Heuristic	Số loại phản hồi	8	5.308	78.632 giây
Kết hợp	Thuật toán di truyền	7	5.250	97.130 giây
Kết hợp	Hàm Step tối ưu	7	5.243	95.562 giây

Thời gian của các chiến lược kết hợp không tính thời gian dùng thuật toán di truyền hoặc điều chỉnh thủ công để tìm tham số; phần đó trong bài gốc mất nhiều giờ.

So sánh theo chiều dọc cho thấy heuristic có hiệu quả tổng hợp tốt nhất. Ưu điểm lớn nhất là hàm đánh giá làm thuật toán có vẻ “biết suy luận” hơn: nó kết hợp khả năng tính toán mạnh của máy tính với một mô hình đánh giá gần với cách con người cân nhắc thông tin. Ngoài ra, khi tối ưu tính $\mathcal{S}[a][b]$, cả mục tối ưu lớp tương đương và mục nhiều vị trí đều dùng cùng một kỹ thuật quan trọng: chia trường hợp. Dù không làm giảm bậc độ phức tạp trong mọi mô hình, kỹ thuật này cải thiện hiệu quả chạy rất nhiều.

Nhìn chung, giải trò chơi đoán số bằng máy tính có thể xem là một bài toán trí tuệ nhân tạo nhỏ: câu hỏi cốt lõi là làm sao để máy tính có năng lực suy luận tốt hơn.

4.2 Hướng nghiên cứu tiếp theo

Với luật chuẩn, vẫn còn không gian cải thiện. Chiến lược tối ưu từng phần hoặc thuật toán di truyền đầy đủ có thể giảm số lượt trung bình ở mức bản chất hơn. Hiệu quả của các chiến lược này trên những luật khác, đặc biệt các biến thể Mastermind, cũng rất đáng nghiên cứu.

Ngoài ra, chiến lược đối kháng được nhắc ở phần tiêu chuẩn thống nhất cũng là một vấn đề lý thuyết trò chơi thú vị: người ra số thay đổi đáp án ngầm nhưng vẫn giữ mọi phản hồi nhất quán. Bài toán đó thậm chí có thể khó hơn chính bài toán giải đoán số.

5 Tài liệu tham khảo

1. Barteld Kooi, University of Groningen. *Yet Another Mastermind Strategy*, 2005.
2. Donald Knuth, Stanford. *The Computer As Master Mind*, 1976.
3. Một lời giải cho trò chơi đoán số: <http://www.cppblog.com/lemene/archive/2007/11/26/37273.html>.
4. Mastermind, Wikipedia: [http://en.wikipedia.org/wiki/Mastermind_\(board_game\)](http://en.wikipedia.org/wiki/Mastermind_(board_game)).
5. Mục từ về trò chơi đoán số trên Baidu Baike.
6. Lời giải bài guess, Winter Camp 2006.

A Phụ lục của bài gốc

Phụ lục bản gốc là phần phản hồi cho môn “Dẫn luận Khoa học và Công nghệ Thông tin”, không liên quan trực tiếp đến thuật toán. Tác giả liệt kê ba giảng viên yêu thích: giáo sư Sun Maosong, viện sĩ Sun Jiaguang và viện sĩ Zhang Bo. Phần góp ý mong nội dung bài giảng hệ thống hơn để sinh viên hiểu rõ khung tổng thể của khoa học và công nghệ thông tin; tác giả cũng đề xuất làm tiểu luận theo nhóm, vì tự

nghiên cứu bài toán đoán số có nhiều ý tưởng nhưng khó hiện thực hết một mình, trong khi năng lực trao đổi và hợp tác nhóm rất quan trọng trong nghiên cứu.